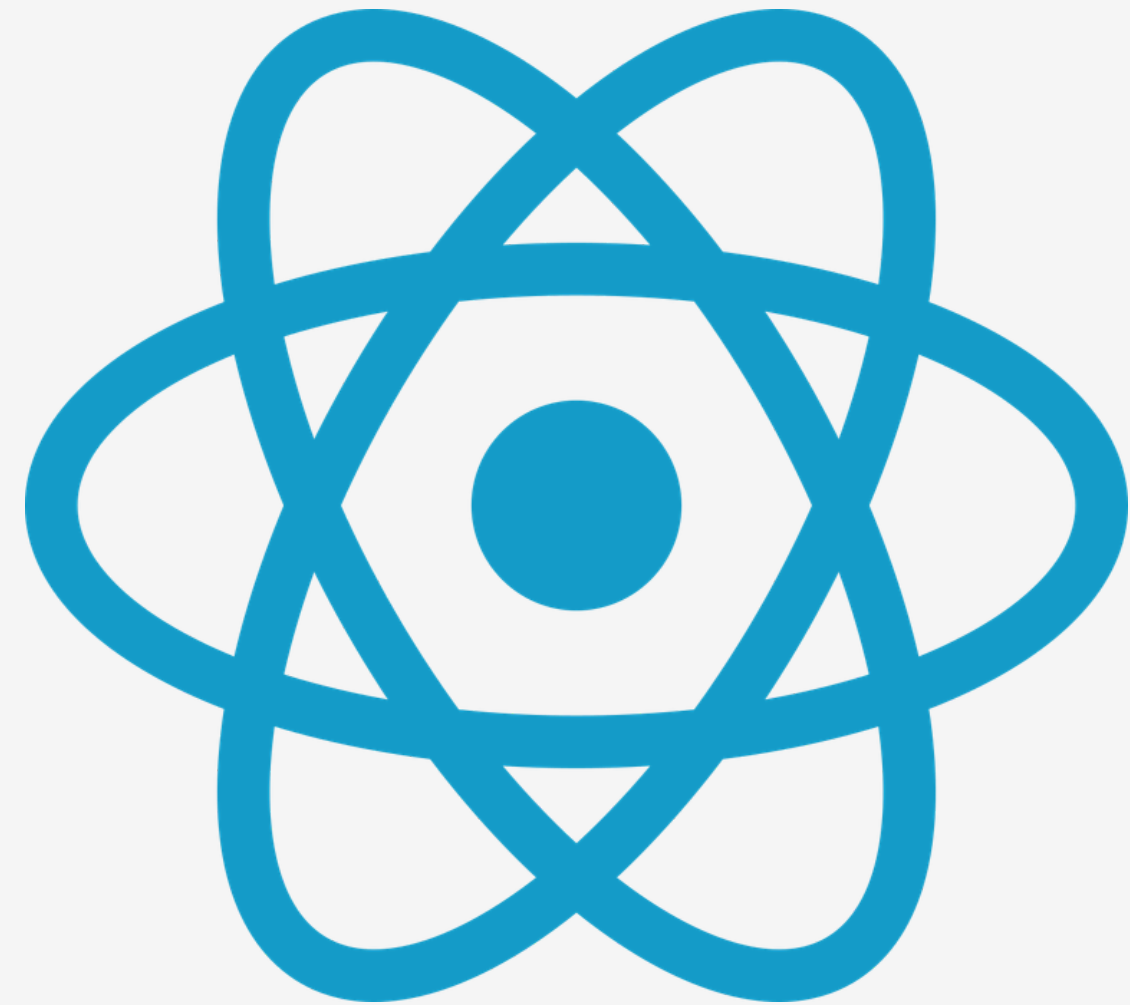


USE CONTEXT+USE REDUCER

Projecte React



Mike Guachamin
2n daw

CONTEXT

PROBLEMA EN APLICACIONES GRANDES

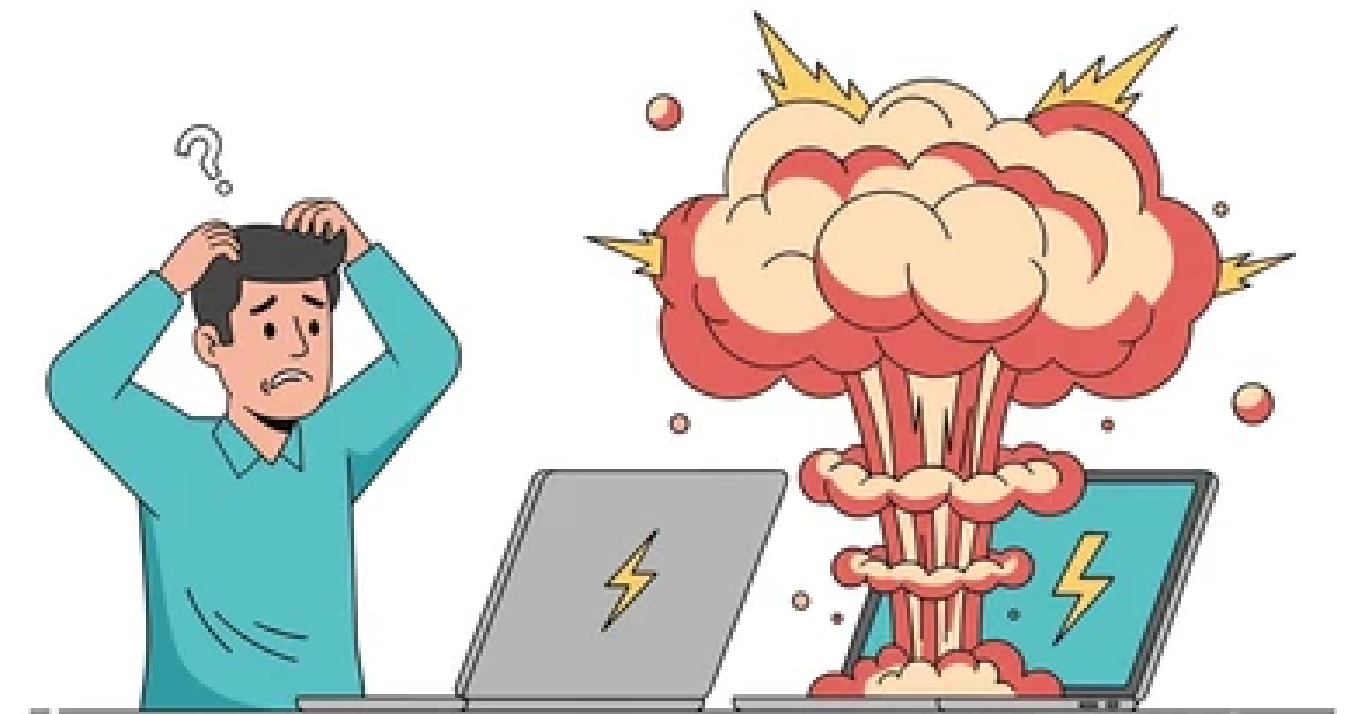
Cuando una aplicación crece, aparecen problemas en la comunicación entre componentes:

- Necesidad de pasar datos a través de muchos componentes, aunque no los utilicen
- Código más largo y difícil de mantener
- Estado desorganizado
- Menor escalabilidad y más riesgo de errores

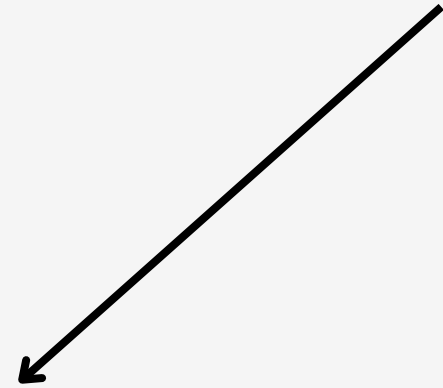


ESTE PROBLEMA SE LLAMA: "PROPS DRILLING"

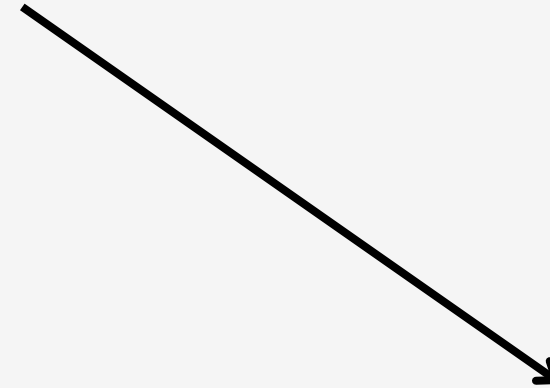
"EL PROPS DRILLING OCURRE CUANDO TENEMOS QUE PASAR DATOS POR MUCHOS COMPONENTES INTERMEDIOS SOLO PARA QUE UN COMPONENTE FINAL PUEDA USARLOS."



SOLUCIÓN



USE CONTEXT



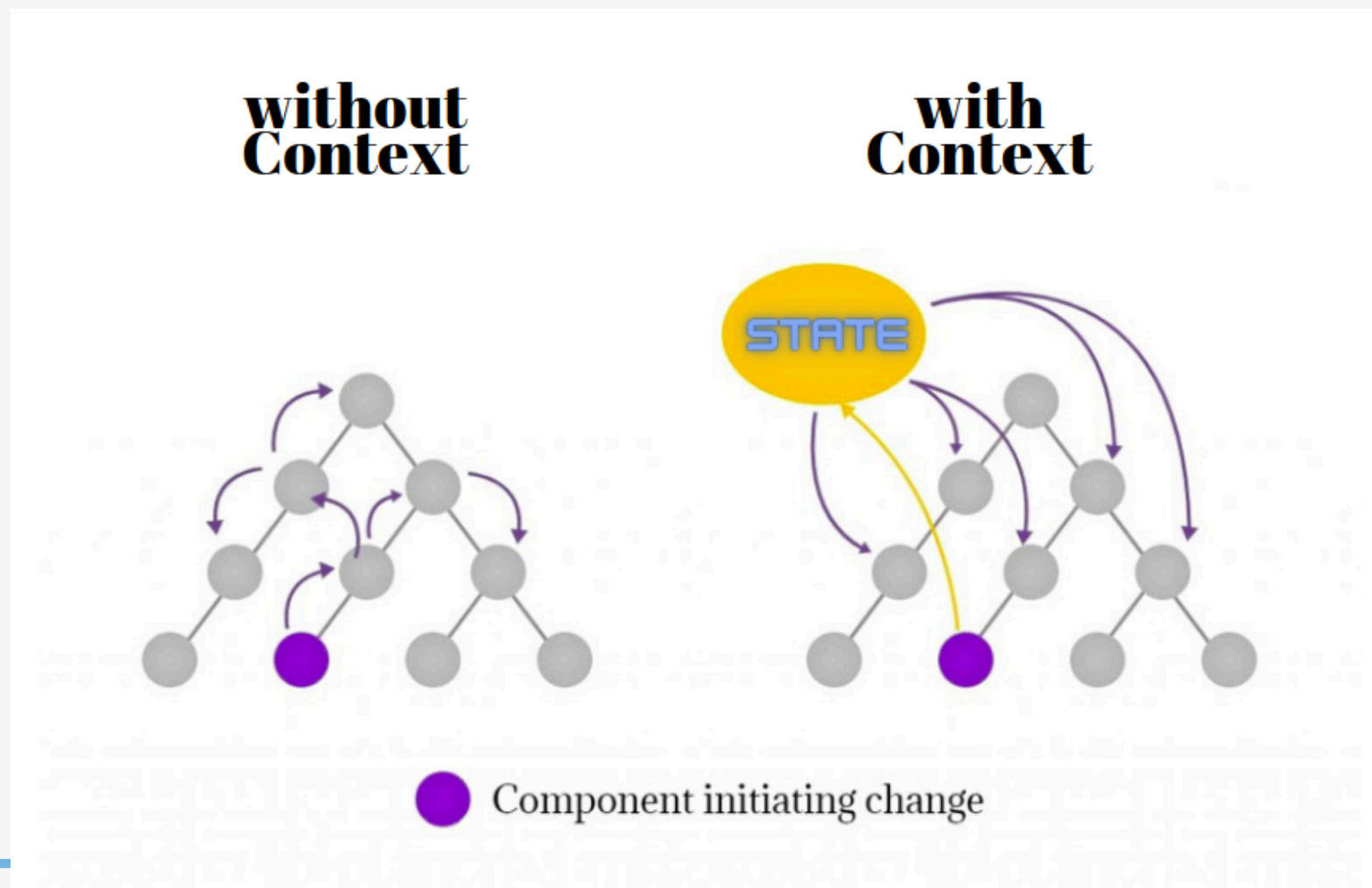
USE REDUCER

**JUNTOS PERMITEN:
GESTIÓN GLOBAL DE ESTADO DE FORMA LIMPIA Y ESCALABLE**



USE CONTEXT

UseContext es un hook de React que permite compartir datos entre componentes sin usar props. Se basa en un Context, que actúa como una fuente de datos global accesible desde cualquier parte de la aplicación.

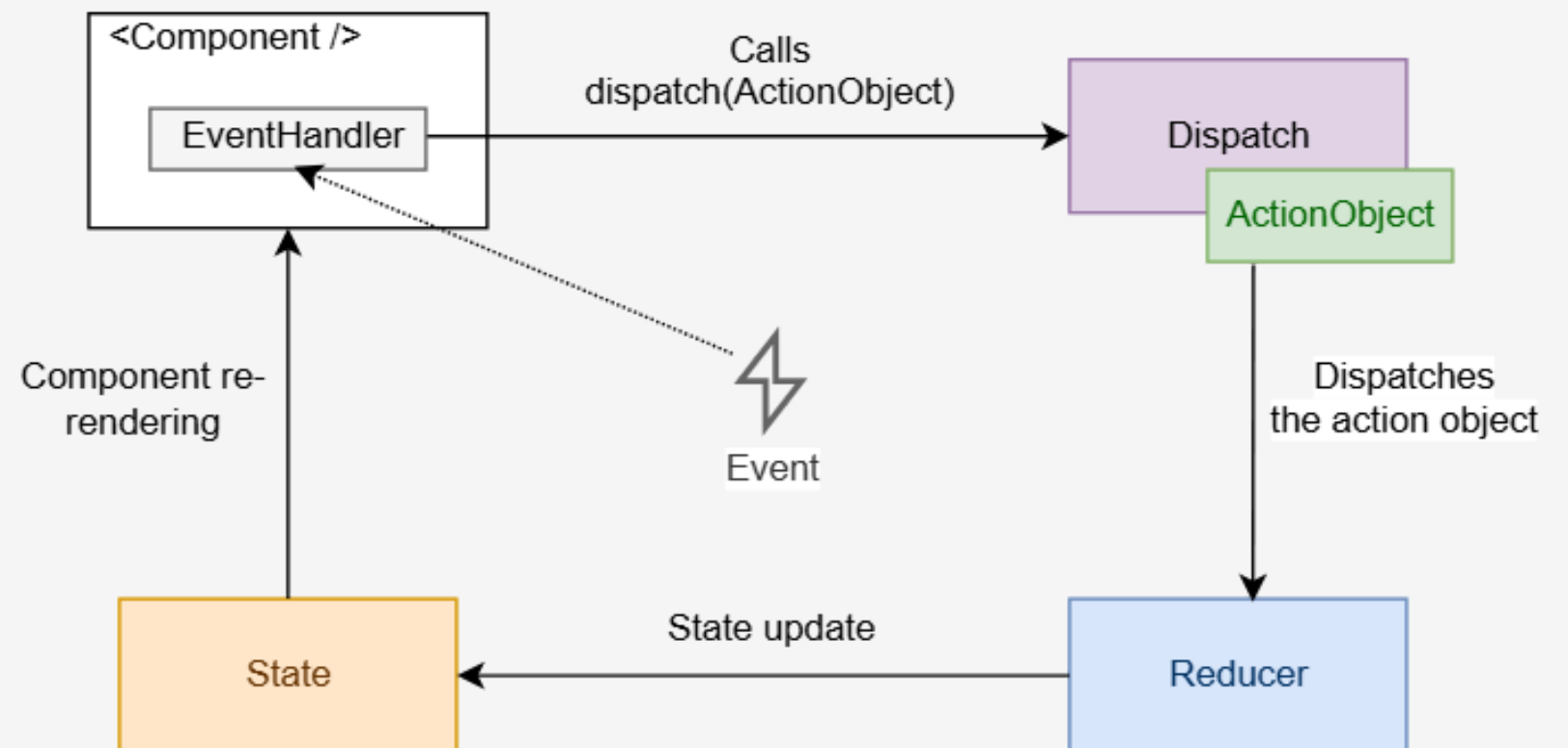


USE REDUCER

useReducer es un hook de React que permite gestionar estados complejos usando una función que controla cómo cambia el estado.

Funciona con:

- estado (state)
- acciones (actions)
- una función (reducer)



DECLARACION



USE CONTEXT

```
const value = useContext(SomeContext)
```

USE REDUCER

```
const [state, dispatch] = useReducer(reducer, initialArg, init?)
```


COMO FUNCIONA CADA UNO?

USE CONTEXT

1. CREAR EL CONTEXT
ES COMO CREAR UN
"CONTENEDOR GLOBAL DE
DATOS"

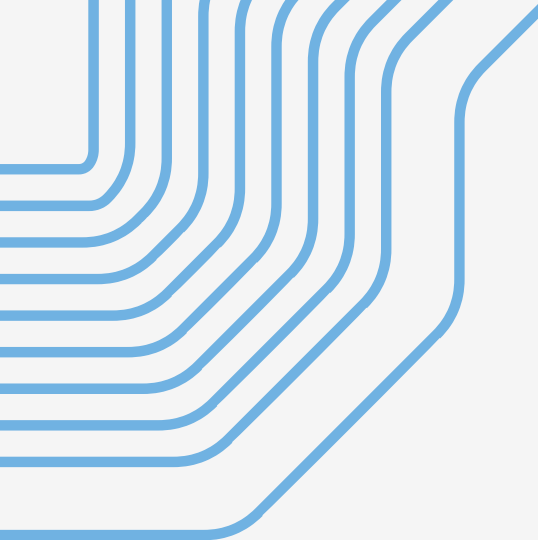
```
import { createContext } from "react";  
  
const MiContext = createContext();
```

ESTO NORMALMENTE SE HACE EN
UN ARCHIVO APARTE.

2. PROVIDER (DONDE GUARDAS LOS DATOS)
AQUÍ DECIDES QUÉ DATOS QUIERES
COMPARTIR:

```
function MiProvider({ children }) {  
  const usuario = "Mike";  
  
  return (  
    <MiContext.Provider value={usuario}>  
      {children}  
    </MiContext.Provider>  
  );  
}
```

TODO LO QUE ESTÉ DENTRO PUEDE
ACCEDER A ESOS DATOS.



3. CONSUMIR LOS DATOS (USECONTEXT): EN CUALQUIER COMPONENTE:

```
import { useContext } from "react";  
  
const usuario = useContext(MiContext);
```

Y YA TIENES EL DATO DIRECTAMENTE.

RESUMEN

1. CREAR EL CONTEXT
 2. CREAR EL PROVIDER (DATOS A COMPARTIR)
 3. ENVOLVER LA APP CON EL PROVIDER
 4. USAR USECONTEXT EN CUALQUIER COMPONENTE
- 

USEREDUCER

1. REDUCER (LA FUNCIÓN PRINCIPAL)
ES LA QUE DECIDE CÓMO CAMBIA EL
ESTADO:

```
function reducer(state, action) {  
  switch (action.type) {  
    case "INCREMENT":  
      return state + 1;  
  
    case "DECREMENT":  
      return state - 1;  
  
    default:  
      return state;  
  }  
}
```

AQUÍ SE DEFINE LA LÓGICA DEL
ESTADO.

2. USEREDUCER (DECLARACIÓN)
SE USA DENTRO DEL COMPONENTE:

```
const [state, dispatch] = useReducer(reducer, 0);
```

STATE → EL VALOR ACTUAL

DISPATCH → LA FUNCIÓN PARA CAMBIAR EL ESTADO

0 → VALOR INICIAL

3. USAR DISPATCH EN LA UI

```
<button onClick={() => dispatch({ type: "SUMAR" })}>+
```

```
</button>
```

```
<button onClick={() => dispatch({ type: "RESTAR" })}>-
```

```
</button>
```

```
<button onClick={() => dispatch({ type: "RESET" })}>↺
```

```
</button>
```

RESUMEN

1. CREAR REDUCER (LÓGICA)
2. USAR USEREDUCER EN EL COMPONENTE
3. USAR DISPATCH PARA CAMBIAR EL ESTADO

USE CONTEXT Y USE REDUCER EN JS VANILLA

En React, useContext comparte datos globales.

En JS puro eso se hace con una variable global o un objeto global.

USECONTEXT EN REACT = OBJETO GLOBAL EN JAVASCRIPT

```
const AppContext = {  
  usuario: "Mike",  
  tema: "dark"  
};
```

```
console.log(AppContext.usuario);
```

ESTO SERÍA COMO EL "CONTEXTO GLOBAL"

useReducer en React controla cambios de estado con acciones.
En JS puro se hace con una función que actualiza una variable.

```
let state = 0;

function reducer(action) {
  switch (action.type) {
    case "SUMAR":
      state = state + 1;
      break;

    case "RESTAR":
      state = state - 1;
      break;

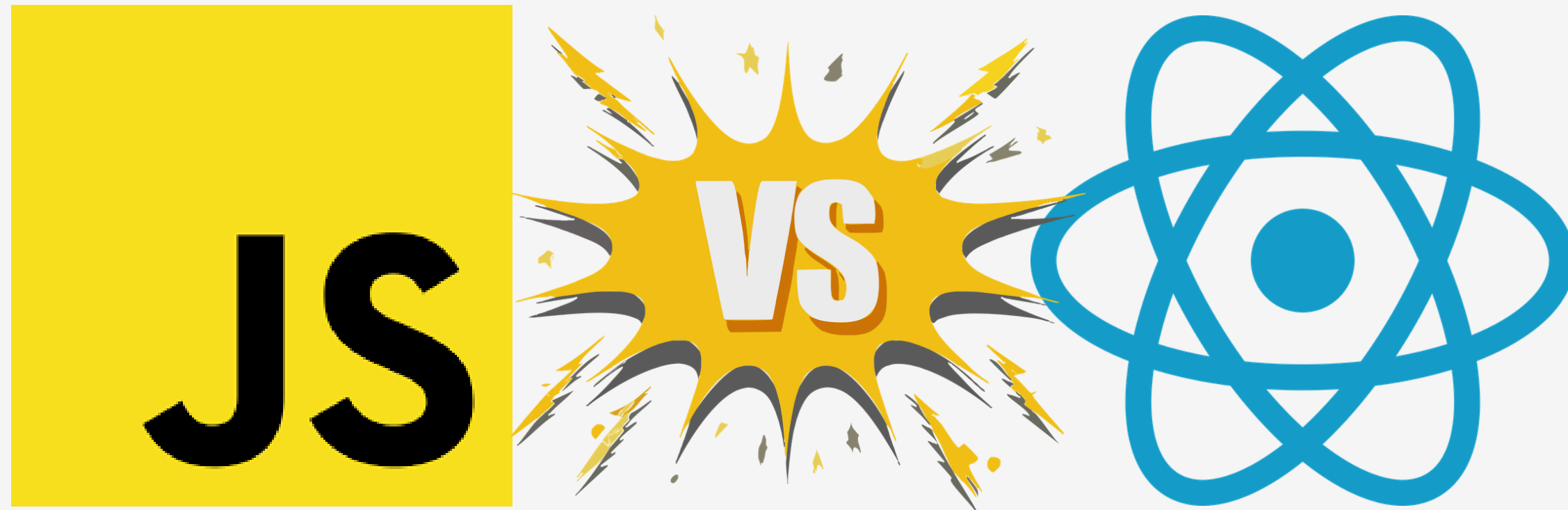
    case "RESET":
      state = 0;
      break;
  }

  console.log(state);
}
```

USEREDUCER EN REACT = FUNCIÓN QUE CONTROLA CAMBIOS DE ESTADO
EN JS PURO = FUNCIÓN QUE MODIFICA VARIABLES MANUALMENTE

CONCLUSION

useContext y useReducer son conceptos de React, pero en JavaScript puro se pueden simular con variables globales y funciones que gestionan el estado manualmente



EJEMPLO PRACTICO

